TCP:
TCP or the **Transmission Control Protocol**, which belongs in the Transport Layer of the Internet Protocol Suit, assures reliability and the ordered delivery of information (in the form of byte streams) from one computer to another. Most of the Internet applications that require reliable and secure data transferring such as World Wide Web, E-mail, peer-to-peer file Sharing, Streaming media applications and other file transferring services, uses TCP for transmission and communication purposes.

IP:
IP or the **Internet Protocol** is the basic protocol that makes up the Internet, as it is responsible for the addressing hosts (computers) and transportation of data packets between hosts, through a packet switched internetwork. Residing on the Internet Layer of Internet Protocol Suite, IP only carries out the task of delivering packets of data (Data grams) from one host to another, depending on the host addresses; therefore, is considered unreliable, as Data Packets send through Internet using IP can be lost, corrupted or delivered in an unordered manner.

IP address:
An IP address, short for **Internet Protocol address**, is an identifying number for network hardware connected to a network. Having an IP address allows a device to communicate with other devices over an IP-based network like the internet.

TCP/IP configuration files

This File	Contains the Following
`/etc/hosts`	IP addresses and host names for your local network as well as any other systems that you access often
`/etc/networks`	Names and IP addresses of networks
`/etc/host.conf`	Instructions on how to translate host names into IP addresses
`/etc/resolv.conf`	IP addresses of name servers
`/etc/hosts.allow`	Instructions on which systems can access Internet services on your system
`/etc/hosts.deny`	Instructions on which systems must be denied access to Internet services on your system
`/etc/nsswitch.conf`	Instructions on how to translate host names into IP addresses

The **`ping`** command is a simple command-line utility in Linux and other operating systems that is used to test the connectivity between two networked devices.

When you run the **`ping`** command, it sends a series of packets to the target device and measures the time it takes for each packet to be sent and received. It also reports any packet loss or errors.

```
ping google.com
```

This command sends a series of packets to **`google.com`** and reports the round-trip time and any packet loss or errors.

```
ping -i 1 google.com
```

This command sends packets to **`google.com`** with an interval of 1 second between each packet.

```
ping -t google.com
```

This command sends packets to **`google.com`** continuously until you stop the command with **`Ctrl + C`**.

The **`ping`** command is a simple and useful tool for testing network connectivity and diagnosing network issues. However, it is important to note that some networks may block or restrict **`ping`** traffic, so it may not always be a reliable method for testing connectivity.

The **`route`** command in Linux and other Unix-based operating systems is used to view and modify the kernel's IP routing table. The routing table is used by the system to determine the path that network traffic takes from the source to the destination.

```
route -n
```

This command displays the current routing table in numeric format.

```
sudo route add -net 192.168.1.0 netmask 255.255.255.0 gw 192.168.0.1
```

This command adds a new route to the **`192.168.1.0`** network with a netmask of **`255.255.255.0`** and a gateway of **`192.168.0.1`**.

```
sudo route del -net 192.168.1.0 netmask 255.255.255.0
```

This command deletes the route to the **`192.168.1.0`** network with a netmask of **`255.255.255.0`**.

```
sudo route add default gw 192.168.0.1
```

This command sets the default gateway to **`192.168.0.1`**.

The **`netstat`** command in Linux is used to display information about the network connections, routing tables, network interfaces and related network statistics. It can be used to diagnose network issues, identify network services that are running and identify potential security threats.

```
netstat -a
```

This command displays all active network connections (both incoming and outgoing).

```
netstat -l
```

This command displays all network services that are currently listening for incoming connections.

```
netstat -s
```

This command displays detailed statistics for each network protocol (such as TCP, UDP, ICMP).

The **`netstat`** command is a powerful tool for network diagnosis and troubleshooting, but it should be used with caution, as incorrect usage can result in inaccurate or misleading information.

`ifconfig` is a command-line utility in Linux and Unix-based operating systems that allows you to configure and display network interfaces.

When you run the **`ifconfig`** command, it displays information about all the active network interfaces on your system, including their IP addresses, netmasks, and hardware (MAC) addresses.

```
Ifconfig
```

This command displays information about all active network interfaces, including their IP addresses, netmasks, and hardware (MAC) addresses.

```
sudo ifconfig eth0 up
```

This command brings the **`eth0`** network interface up, enabling it to send and receive data. To bring it down, replace **`up`** with **`down`**

```
sudo ifconfig eth0 192.168.1.100 netmask 255.255.255.0
```

This command assigns the IP address **`192.168.1.100`** and netmask **`255.255.255.0`** to the **`eth0`** network interface.

```
ifconfig eth0
```

This command displays information about only the **`eth0`** network interface.

The **tcpdump** command in Linux is a network packet analyzer that allows you to capture and analyze network traffic in real-time or from a saved file. It can be used for network troubleshooting, monitoring, and security analysis.

```
sudo tcpdump -i eth0
```

This command captures all network traffic on the **eth0** interface.

```
sudo tcpdump -i eth0 port http
```

This command captures only HTTP traffic on the **eth0** interface.

```
sudo tcpdump -i eth0 host 192.168.0.1
```

This command captures only traffic to or from the IP address **192.168.0.1** on the **eth0** interface.

```
sudo tcpdump -i eth0 -w capture.pcap
```

This command captures all traffic on the **eth0** interface and saves it to a file called **capture.pcap**

```
tcpdump -r capture.pcap
```

This command reads and analyzes the contents of the **capture.pcap** file.

The **tcpdump** command provides a wide range of options and filters that allow you to capture and analyze network traffic in a variety of ways. However, it should be used with caution, as capturing and analyzing network traffic can potentially reveal sensitive information, such as usernames and passwords.

NIS & NFS

- The Network Information Service (NIS) and Network File System (NFS) are services that allow you to build distributed computing systems that are both consistent in their appearance and transparent in the way files and data are shared.
- NFS and NIS are high-level networking protocols, built on several lower-level protocols.

Network Information Service (NIS)

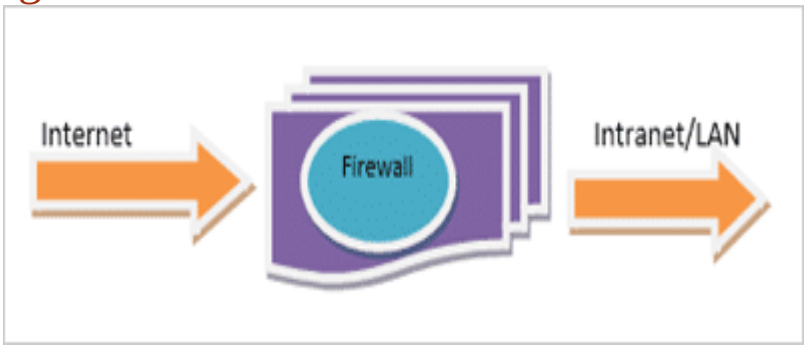
- NIS provides a distributed database system for common configuration files.
- NIS servers manage copies of the database files, and NIS clients request information from the servers instead of using their own, local copies of these files. For example, the */etc/hosts* file is managed by NIS.
- A few NIS servers manage copies of the information in the hosts file, and all NIS clients ask these servers for host address information instead of looking in their own */etc/hosts* file.
- Once NIS is running, it is no longer necessary to manage every */etc/hosts* file on every machine in the network — simply updating the NIS servers ensures that all machines will be able to retrieve the new configuration file information.

Network File System (NFS)

- NFS is a distributed file system.
- An NFS server has one or more file systems that are mounted by NFS clients; to the NFS clients, the remote disks look like local disks.
- NFS file systems are mounted using the standard Unix *mount* command, and all Unix utilities work just as well with NFS-mounted files as they do with files on local disks.
- NFS makes system administration easier because it eliminates the need to maintain multiple copies of files on several machines: all NFS clients share a single copy of the file on the NFS server.
- NFS also makes life easier for users: instead of logging on to many different systems and moving files from one system to another, a user can stay on one system and access all the files that he or she needs within one consistent file tree.

Firewall

- The concept of the firewall was introduced to secure the communication process between various networks.
- A firewall is a software or a hardware device that examines the data from several networks and then either permits it or blocks it to communicate with your network and this process is governed by a set of predefined security guidelines.
- A firewall is a device or a combination of systems that supervises the flow of traffic between distinctive parts of the network. A firewall is used to guard the network against nasty people and prohibit their actions at predefined boundary levels.
- A firewall is not only used to protect the system from exterior threats but the threat can be internal as well. Therefore we need protection at each level of the hierarchy of networking systems.
- A good firewall should be sufficient enough to deal with both internal and external threats and be able to deal with malicious software such as worms from acquiring access to the network. It also provisions your system to stop forwarding unlawful data to another system.
- a firewall always exists between a private network and the Internet which is a public network thus filters packets coming in and out.



Selecting a precise firewall is critical in building up a secure networking system.

Firewall provisions the security apparatus for allowing and restricting traffic, authentication, address translation, and content security.
It ensures 365 *24*7 protection of the network from hackers. It is a one-time investment for any organization and only needs timely updates to function properly.
By deploying a firewall there is no need for any panic in case of network attacks.

How does a firewall work?

- A firewall establishes a border between an external network and the network it guards. It is inserted inline across a network connection and inspects all packets entering and leaving the guarded network. As it inspects, it uses a set of pre-configured rules to distinguish between benign and malicious packets.
- The term 'packets' refers to pieces of data that are formatted for internet transfer. Packets contain the data itself, as well as information about the data, such as where it came from. Firewalls can use this packet information to determine whether a given packet abides by the rule set. If it does not, the packet will be barred from entering the guarded network.
- Rule sets can be based on several things indicated by packet data, including:

Their source, Their destination, Their content.

Types of Firewall

1. Software Firewall :

Software firewall is a special type of computer software runs on a computer/server. It's main purpose is to protect your computer/server from outside attempts to control or gain access and depending on your choice of software firewall. Software firewall can also be configured for checking any suspicious outgoing requests.

2. Hardware Firewall :

It is physical piece of equipment planned to perform firewall duties. A hardware firewall can be a computer or a dedicated piece of equipment which serve as a firewall. Hardware firewall are incorporated into the router that is situated between the computer and the internet gateway.

Types of Software Firewall

- **Packet Filtering Firewalls**

Packet filtering firewalls are the oldest, most basic type of firewalls. Operating at the network layer, they check a data packet for its source IP and destination IP, the protocol, source port, and destination port against predefined rules to determine whether to pass or discard the packet.

- **Circuit-Level Gateways**

Working at the session layer, circuit-level gateways verify established Transmission Control Protocol (TCP) connections and keep track of the active sessions. Primarily, they determine the security of an established connection. When an internal device initiates a connection with a remote host, circuit-level gateways establish a virtual connection on behalf of the internal device to keep the identity and IP address of the internal user hidden.

- **Stateful Inspection Firewalls**

stateful inspection firewalls, and verifying and keeping track of established connections also perform packet inspection to provide better, more comprehensive security. They work by creating a state table with source IP, destination IP, source port, and destination port once a connection is established. They create their own rules dynamically to allow expected incoming network traffic instead of relying on a hardcoded set of rules based on this information. They conveniently drop data packets that do not belong to a verified active connection.

- **Application-Level Gateways (Proxy Firewalls)**

Application-level gateways, also known as proxy firewalls, are implemented at the application layer via a proxy device. Instead of an outsider accessing your internal network directly, the connection is established through the proxy firewall. The external client sends a request to the proxy firewall. After verifying the authenticity of the request, the proxy firewall forwards it to one of the internal devices or servers on the client's behalf. Alternatively, an internal device may request access to a webpage, and the proxy device will forward the request while hiding the identity and location of the internal devices and network.

Designing a Firewall

- iptables - administration tool for IPv4 packet filtering and NAT
- **Iptables** is used to set up, maintain, and inspect the tables of IP packet filter rules in the Linux kernel. Several different tables may be defined. Each table contains a number of built-in chains and may also contain user-defined chains.
- Each chain is a list of rules which can match a set of packets. Each rule specifies what to do with a packet that matches. This is called a 'target', which may be a jump to a user-defined chain in the same table.
- A firewall rule specifies criteria for a packet, and a target. If the packet does not match, the next rule in the chain is the examined; if it does match, then the next rule is specified by the value of the target, which can be the name of a user-defined chain or one of the special values *ACCEPT*, *DROP*, *QUEUE*, or *RETURN*.
- *ACCEPT* means to let the packet through. *DROP* means to drop the packet on the floor. *QUEUE* means to pass the packet to userspace. *RETURN* means stop traversing this chain and resume at the next rule in the previous (calling) chain.
- https://linux.die.net/man/8/iptables
- https://www.youtube.com/watch?v=vbhr4csDeI4
- https://www.youtube.com/watch?v=6Ra17Qpj68c

`iptables` is a command-line utility that is used to manage the netfilter firewall in Linux. It is used to set up, maintain, and inspect the rules for the firewall. **`iptables`** is an extremely powerful tool that provides fine-grained control over the incoming and outgoing network traffic on a Linux system.

```
sudo iptables -L
```

This command displays a list of all the current firewall rules.

Sample output:

```
Chain INPUT (policy DROP)
target     prot opt source               destination
ACCEPT     all  --  anywhere             anywhere             ctstate RELATE
ACCEPT     tcp  --  anywhere             anywhere             tcp dpt:ssh
ACCEPT     tcp  --  anywhere             anywhere             tcp dpt:http
ACCEPT     tcp  --  anywhere             anywhere             tcp dpt:https
DROP       all  --  anywhere             anywhere

Chain FORWARD (policy DROP)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
```

This output shows the current **`iptables`** rules that are in effect on the system. The **`INPUT`** chain is responsible for filtering incoming traffic, while the **`OUTPUT`** chain is responsible for filtering outgoing traffic.

In this example, the policy for the **`INPUT`** chain is set to **`DROP`**, which means that all incoming traffic is dropped by default unless there is a matching rule that explicitly allows it. The **`ACCEPT`** rules allow incoming SSH, HTTP, and HTTPS traffic, while the **`DROP`** rule at the end of the chain drops all other incoming traffic.

The **`OUTPUT`** chain has a policy of **`ACCEPT`**, which means that all outgoing traffic is allowed by default unless there is a matching rule that explicitly blocks it.

```
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
```

This command allows incoming traffic on port 22 (SSH) by adding a new rule to the INPUT chain.

```
sudo iptables -A INPUT -s 192.168.0.1 -j DROP
```

This command blocks all incoming traffic from the IP address `192.168.0.1` by adding a new rule to the INPUT chain.

Design a firewall using iptables that incoming SSH and HTTP traffic, and outgoing traffic on all ports. All other traffic will be dropped.

Step 1: First, we'll set the default policies for the INPUT, FORWARD, and OUTPUT chains to DROP. This means that any traffic that doesn't match a specific rule will be dropped by default.

```
sudo iptables -P INPUT DROP
sudo iptables -P FORWARD DROP
sudo iptables -P OUTPUT DROP
```

Step 2: Allow incoming traffic on port 22 (SSH) and port 80 (HTTP).

```
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
```

Step 3: Allow outgoing traffic on all ports.

```
sudo iptables -A OUTPUT -j ACCEPT
```

With these rules in place, our firewall will allow incoming SSH and HTTP traffic, and outgoing traffic on all ports. All other traffic will be dropped.

Design a firewall that block that blocks all incoming and out going traffic from website “facebook.com”

```
sudo iptables -A OUTPUT -d www.facebook.com -j DROP
sudo iptables -A INPUT -s www.facebook.com -j DROP
```

Design a firewall that blocks all incoming and outgoing traffic from group of social media websites

```
sudo iptables -A OUTPUT -m string --string "facebook" --algo kmp -j
DROP
sudo iptables -A OUTPUT -m string --string "twitter" --algo kmp -j DROP
sudo iptables -A OUTPUT -m string --string "instagram" --algo kmp -j
DROP
sudo iptables -A OUTPUT -m string --string "linkedin" --algo kmp -j
DROP
sudo iptables -A INPUT -m string --string "facebook" --algo kmp -j DROP
sudo iptables -A INPUT -m string --string "twitter" --algo kmp -j DROP
sudo iptables -A INPUT -m string --string "instagram" --algo kmp -j
DROP
sudo iptables -A INPUT -m string --string "linkedin" --algo kmp -j DROP
```